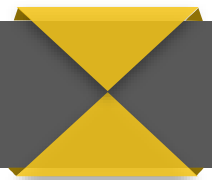


Data Structure and Algorithm

# Search Algorithms

# Outline



- What is search?
- Why need to search?
- Search application
- Basic search algorithms
  - Sequential search
  - Binary search
  - Search using hash table
- String match

# What is search?

- ***Search*** is the process of determining the position of an element with a specified feature in a set.



# Easy or difficult to search?

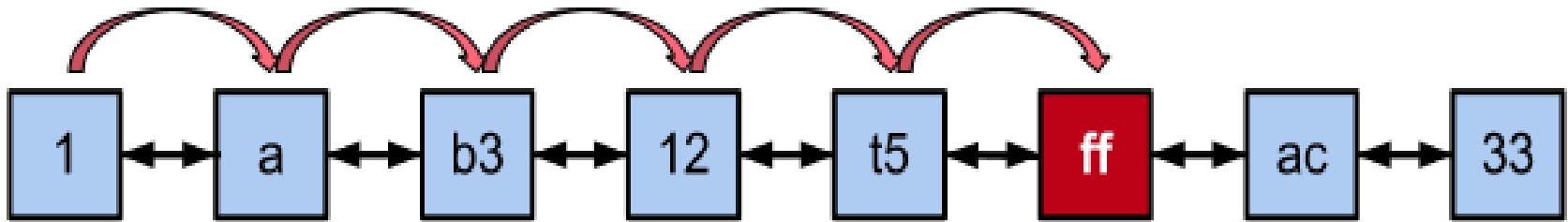
- Find the number 12 in the following array?

|       | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7 | 8 | 9  |
|-------|----|----|----|----|----|----|----|---|---|----|
| Array | 23 | 17 | 97 | 44 | 35 | 10 | 12 | 8 | 5 | 78 |



# Easy or difficult to search?

- Find the string ff in the following linked list?

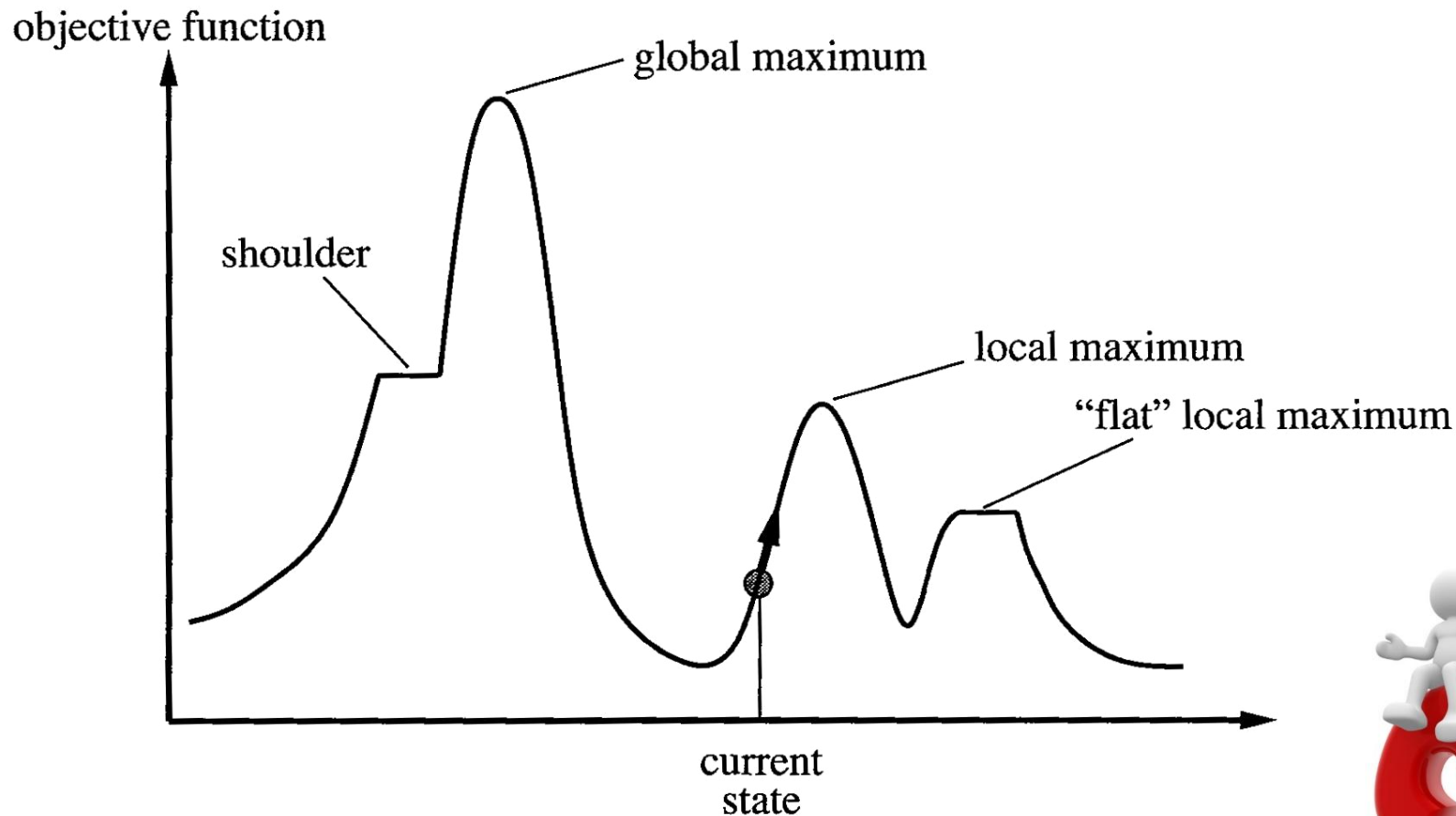


# Quiz

- Let  $S$  be an unsorted array of  $n$  integers. Give an algorithm that finds the pair  $x, y \in S$  that minimizes  $|x - y|$ , for  $x \neq y$ . What is the runtime of your algorithms?

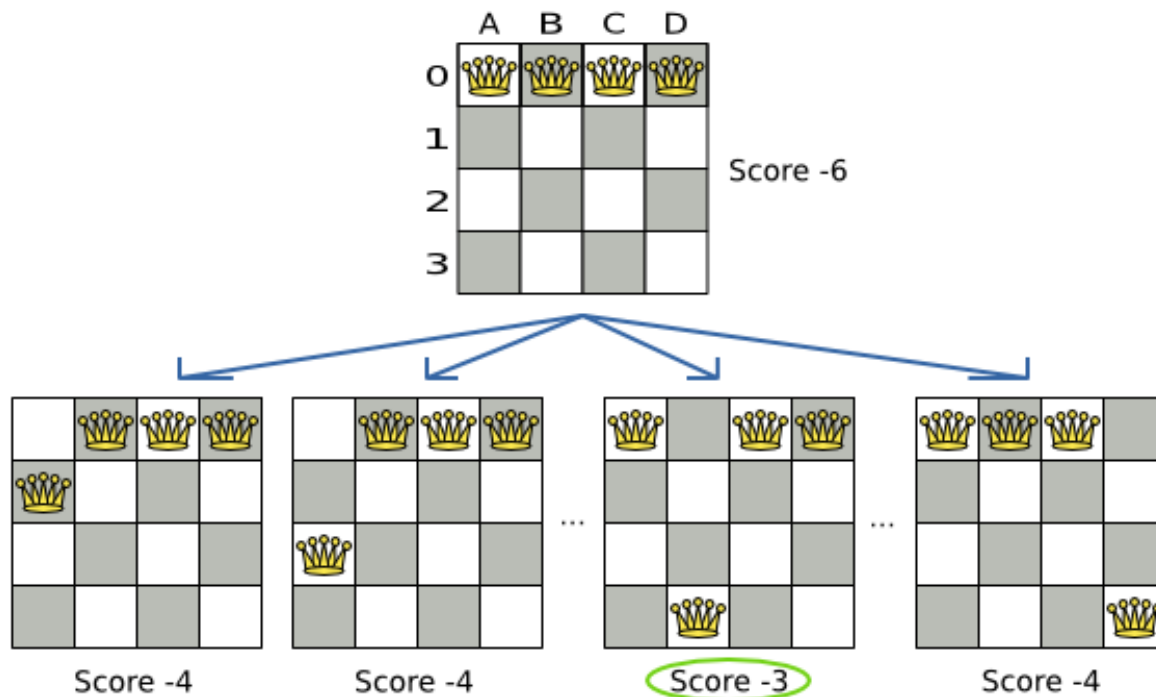
# Easy or difficult to search?

- Determining the highest mountain peak when you are blindfolded?



# Easy or difficult to search?

- Placing eight chess queens on an 8×8 chessboard so that no two queens threaten each other?





# Easy or difficult to search?

- If all of the above is easy then consider this question:
  - Find someone to love "forever"?



# Why?

- Easy because:
  - The power of the computer
  - The power of storage
  - The power of the algorithm
- Difficult because:
  - Big data
  - Time consuming
  - Depends on how to store it
  - The feature is difficult to find or describe
  - Lack of search knowledge: how the brain works to recognize images, ...

# Search application

- Search for the website
  - Google, Yahoo, Bing, ...
- Database
  - Search for a record
- ...

# Simple search algorithm structure

- **Input:**
  - List which consists of  $n$  elements
  - Describe the element to look for
- **Output:**
  - Returns whether or not the element is present
  - Returns the element to be found.
- **Basic operation:**
  - Compare

# Basic search algorithms

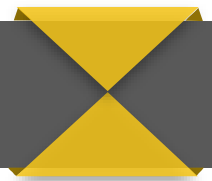
- Linear/sequential search
- Binary search
- Search tree
- Hashing

# Linear/sequential search

- How do I find a house when I know its address?



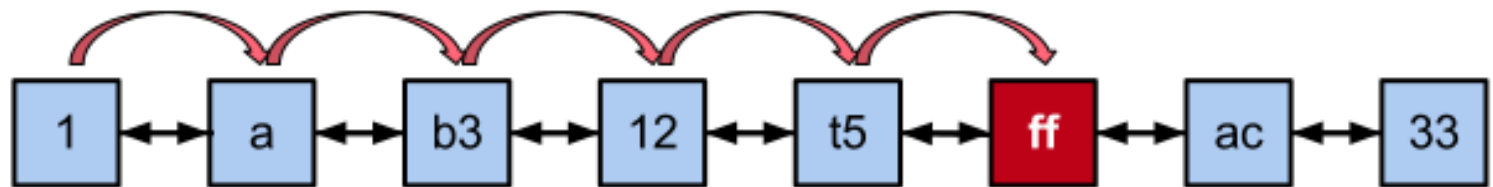
# Linear/sequential search



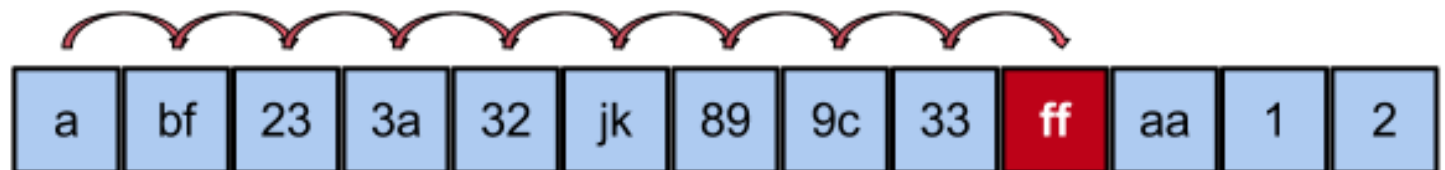
## Idea:

- Compare  $x$  with the elements of the array in turn until it meets the desired element or runs out of the array

### 1. Linked Lists search for "ff"



### 2. Arrays sequential search



# Linear/sequential search

- Example:

| Key | List            |
|-----|-----------------|
| 3   | 6 4 1 9 7 3 2 8 |
| 3   | 6 4 1 9 7 3 2 8 |
| 3   | 6 4 1 9 7 3 2 8 |
| 3   | 6 4 1 9 7 3 2 8 |
| 3   | 6 4 1 9 7 3 2 8 |
| 3   | 6 4 1 9 7 3 2 8 |



# Linear/sequential search

*Pseduo code*

```
for (i ← 0 to n) do  
    if (x = a[i]) then  
        return true;  
end for
```

# Evaluate sequential search

- Advantage:
  - Ease of implementation
- Disadvantage:
  - Long searching time:  $O(n)$

# Sentinel Linear Search

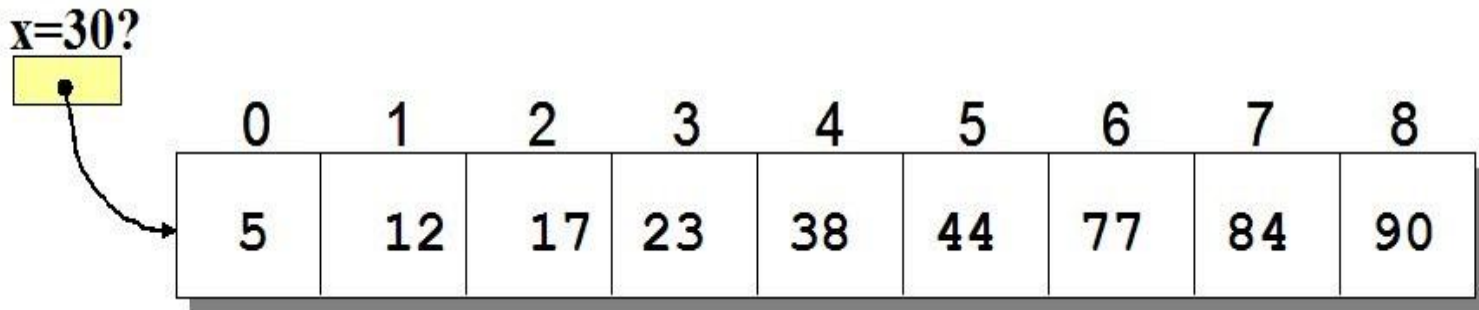
- How many comparisons / tests are there in each sequential search loop?
  - Checks if the element is  $x$
  - Checks out of bounds of the array.
- **Sentinel element**: places the element  $x$  at the end of the array → No need to check out of bounds of the array.
  - Example:  $A = \{1, 25, 5, 2, 37\}$  and  $x = 6$

|   |    |   |   |    |   |
|---|----|---|---|----|---|
| 1 | 25 | 5 | 2 | 37 | 6 |
|---|----|---|---|----|---|

- Return:  $n$  if not found.

# Sequential search on ordinal array

- *If the list is sorted, how good is it for sequential search?*
  - With an ordered array, instead of finding the entire array, we escape when we encounter an element that is larger than the desired one.



# Quiz

- Suppose that you are given a **sorted sequence** of **distinct integers**  $\{a_1, a_2, \dots, a_n\}$ . Give an  $O(\log n)$  algorithm to determine whether there exists an  $i$  index such as  $a_i = i$ .

For example, in  $\{-10, -3, 3, 5, 7\}$ ,  $a_3 = 3$ . In  $\{2, 3, 4, 5, 6, 7\}$ , there is no such  $i$

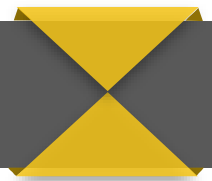
# Binary search

- Once the array is ordered, we take advantage of this to reduce the number of operations.
  - How do you find a new word in a dictionary?



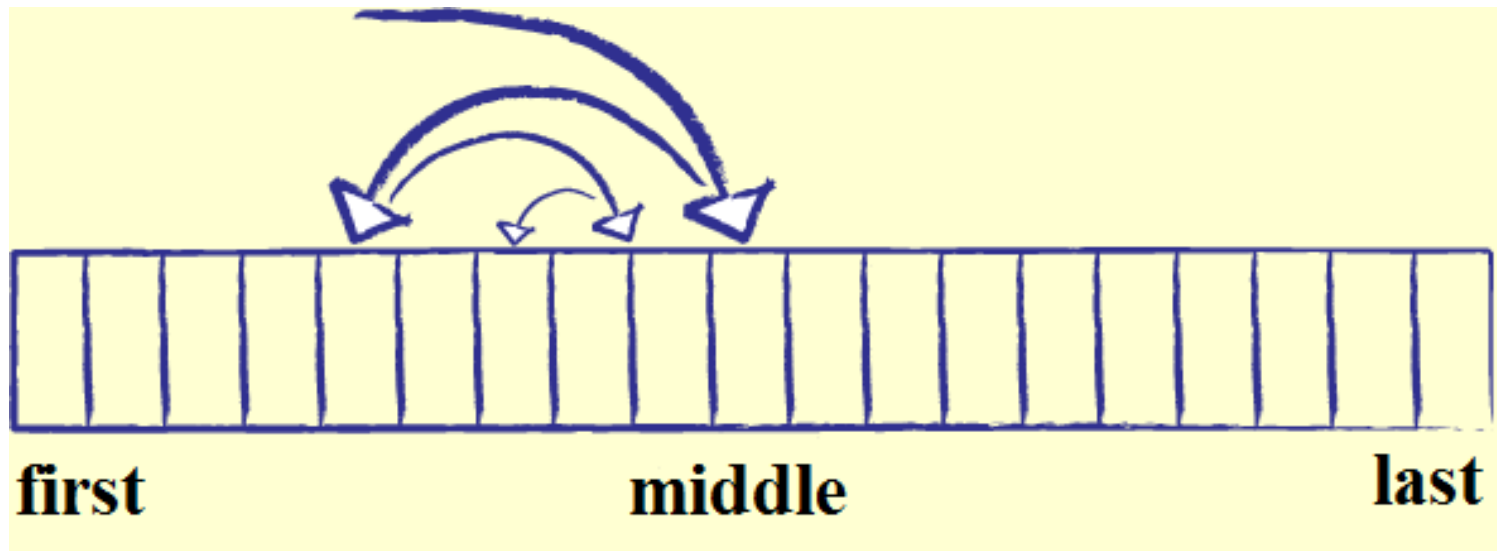
Binary Search

# Binary search



Idea: devide and conquer

- Check the middle element  $a[m]$
- If  $a[m] = x$ , return  $x$
- If  $a[m] > x$ , find  $x$  in the same way in the left sub-sequence
- If  $a[m] < x$ , find  $x$  in the same way in the right sub-sequence

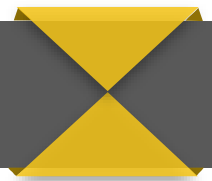




- Diagram illustrating the recursive steps of Merge Sort on the array [2, 6, 7, 34, 76, 123, 234, 567, 677, 986]:
- Initial array: 2 6 7 34 76 123 234 567 677 986. Markers: first (0), mid (4), last (9).
  - Splitting phase:
    - Array: 2 6 7 34 76 123 234 567 677 986. Markers: (5) first, (7) mid, (9) last.
    - Array: 2 6 7 34 76 123 234 567 677 986. Markers: (5) first, mid, last (6).
  - Merging phase:
    - Array: 2 6 7 34 76 123 234 567 677 986. Markers: first mid last (5).



# Binary search



*Pseudo Code*

```
left = 0; right = n - 1;
while (l ≤ r) do
    m = (left + right) / 2;
    if (x = a[m]) then return true; //found
    else
        if (x < a[m]) then
            right = m - 1; //find on left side
        else
            left = m + 1; //find on right side
        end if
    end if
end while
return -1; //not found
```

# Quiz



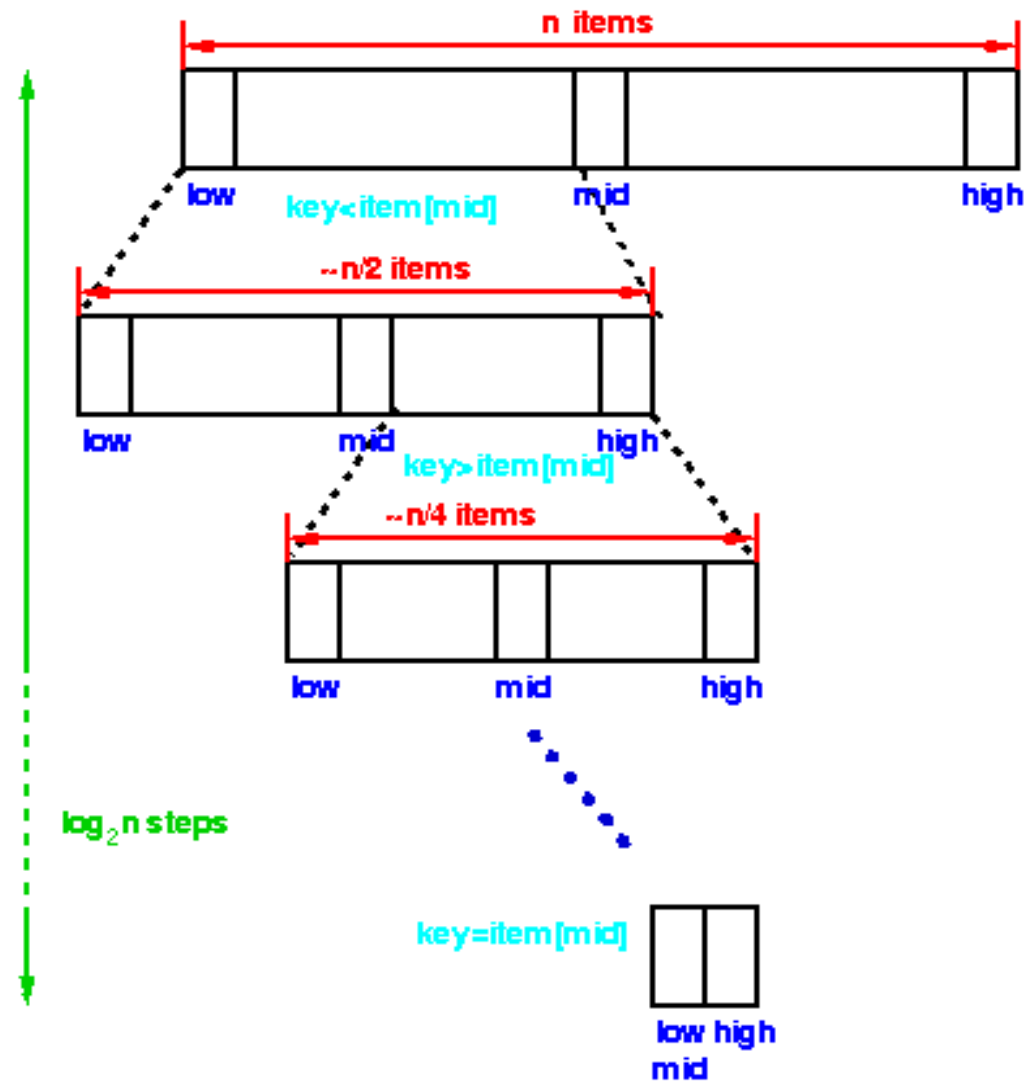
- How many steps would it take to perform a linear search for the number 8 in the ordered array, [2, 4, 6, 8, 10, 12, 13]?
- How many steps would binary search take for the previous example?

# Quiz

- Suppose you are given an array  $A$  of  $n$  sorted numbers that has been circularly shifted  $k$  positions to the right. For example,  $\{35, 42, 5, 15, 27, 29\}$  is a sorted array that has been circularly shifted  $k = 2$  positions, while  $\{27, 29, 35, 42, 5, 15\}$  has been shifted  $k = 4$  positions.
- A) Suppose you know what  $k$  is. Give an  $O(1)$  algorithm to find the largest number in  $A$ .
- B) Suppose you do not know what  $k$  is. Give an  $O(\lg n)$  algorithm to find the largest number in  $A$ . For partial credit, you may give an  $O(n)$  algorithm.

# Evaluate a binary search

- Advantage:
  - Time complexity:  
 $O(\log_2 n)$
- Disadvantage:
  - Arrays must be sorted first.

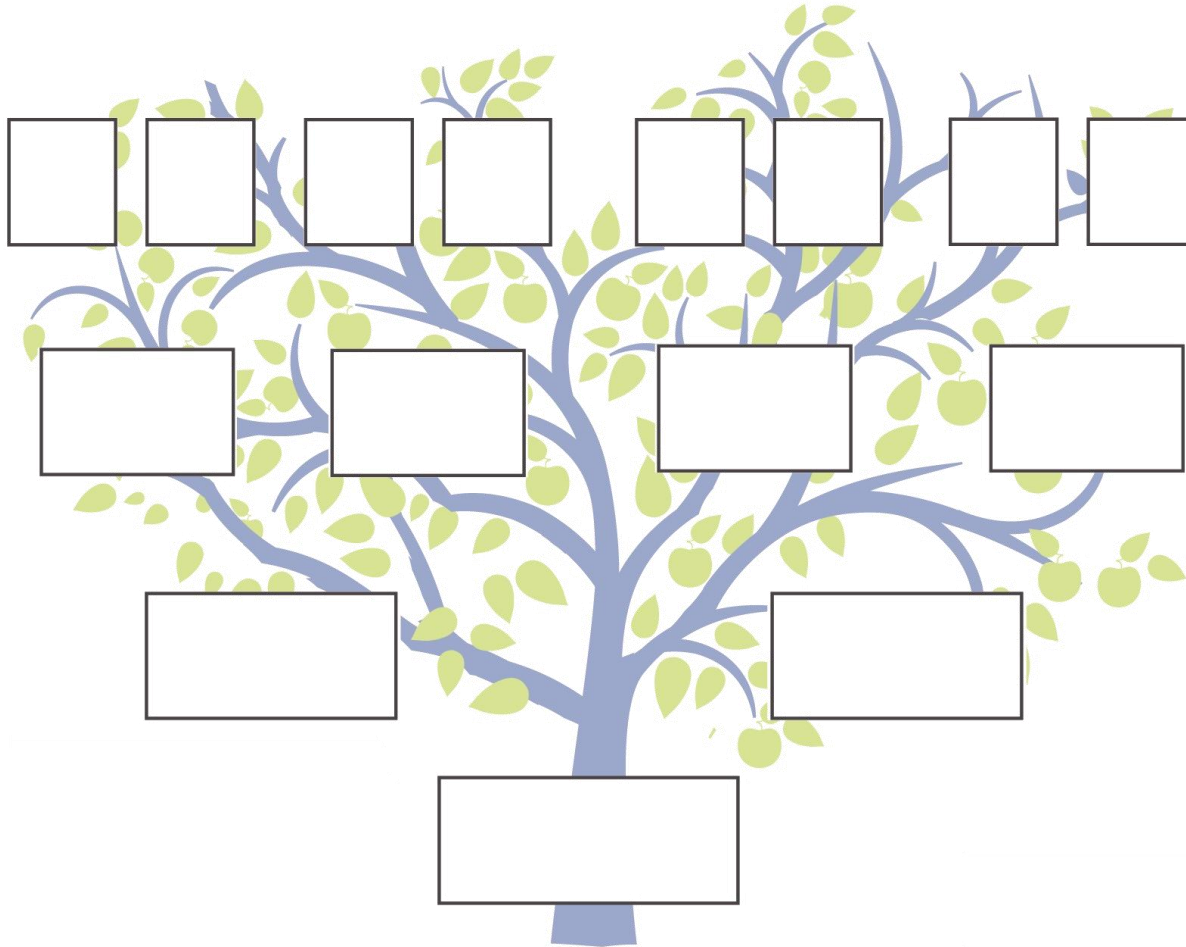


# Basic search algorithms

- Linear/sequential search
- Binary search
- Search tree
- Hashing

# Search tree

Will be introduced in the next lesson



# Quiz

- Write a function that returns the intersection of two arrays. The intersection is a third array that contains all values contained within the first two arrays. For example, the intersection of [1, 2, 3, 4, 5] and [0, 2, 4, 6, 8] is [2, 4]. Your function should have a complexity of  $O(N)$ .

# Quiz

- Write a function that accepts an array of strings and returns the first duplicate value it finds. For example, if the array is ["a", "b", "c", "d", "c", "e", "f"], the function should return "c", since it's duplicated within the array. (You can assume that there's one pair of duplicates within the array.) Make sure the function has an efficiency of  $O(N)$ .

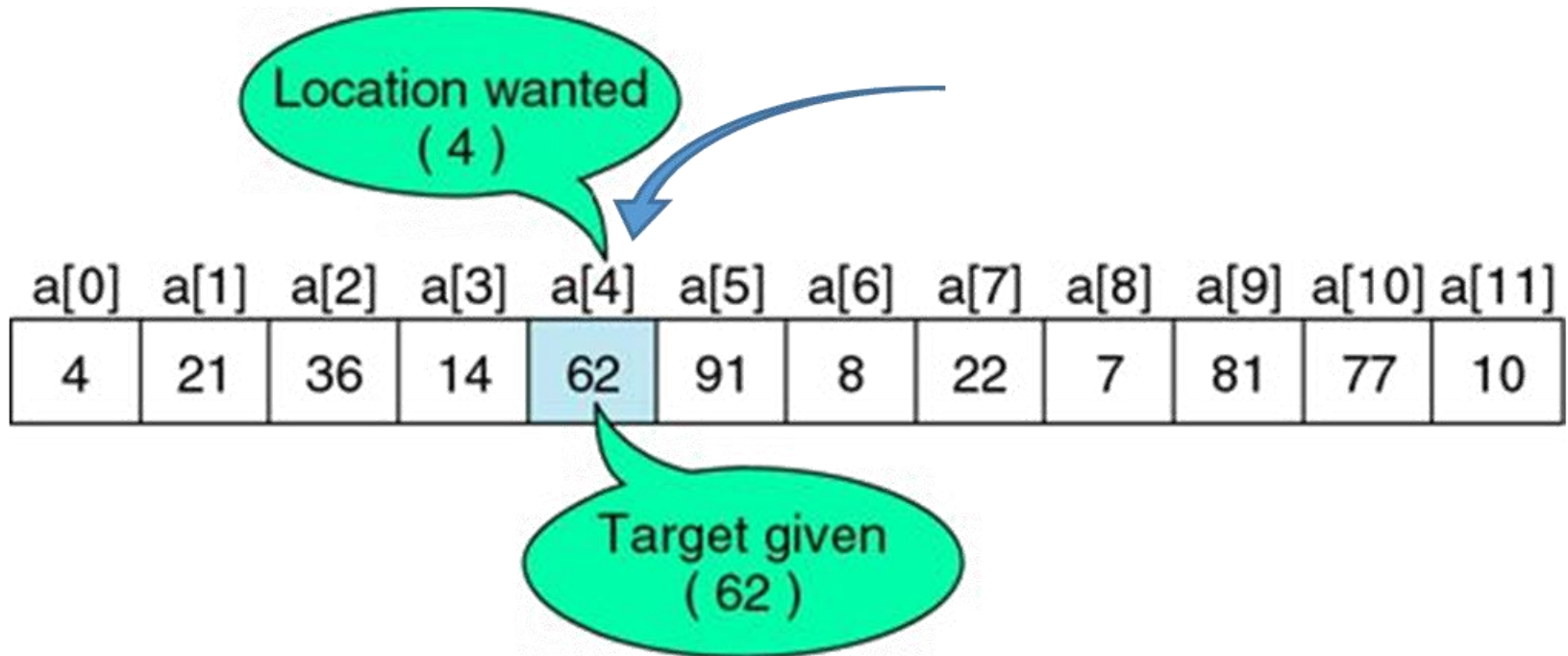


# Basic search algorithms

- Linear/sequential search
- Binary search
- Search tree
- Hashing

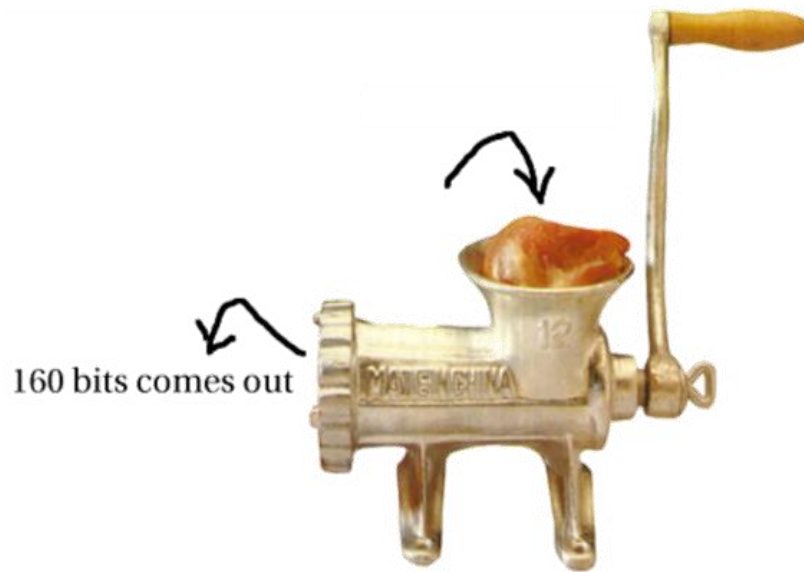
# Hashing

- Is there a way to find the element in just one step?



# Hashing

- **Hashing** is a technique of converting an element into another form to serve a specific purpose such as address, key, ...



Process of meat transformation → minced meat

# Hashing

- **Hash function** is the mapping from one key to another.

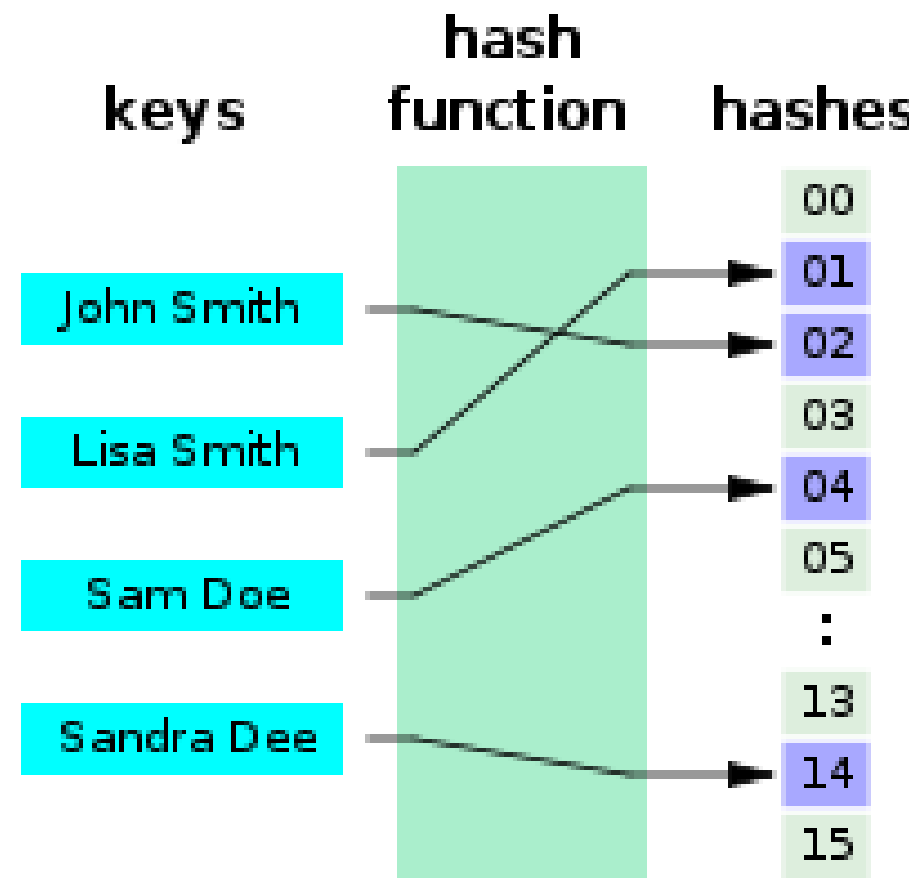
Example:  $H(k) = k \bmod M$

H:  $U \longrightarrow A$

$k \longrightarrow a = H(k)$

For example, suppose the characters are encoded as follows:  
 $c = 3$ ;  $a = 1$ ;  $t = 20$ ;  $s = 19$ .

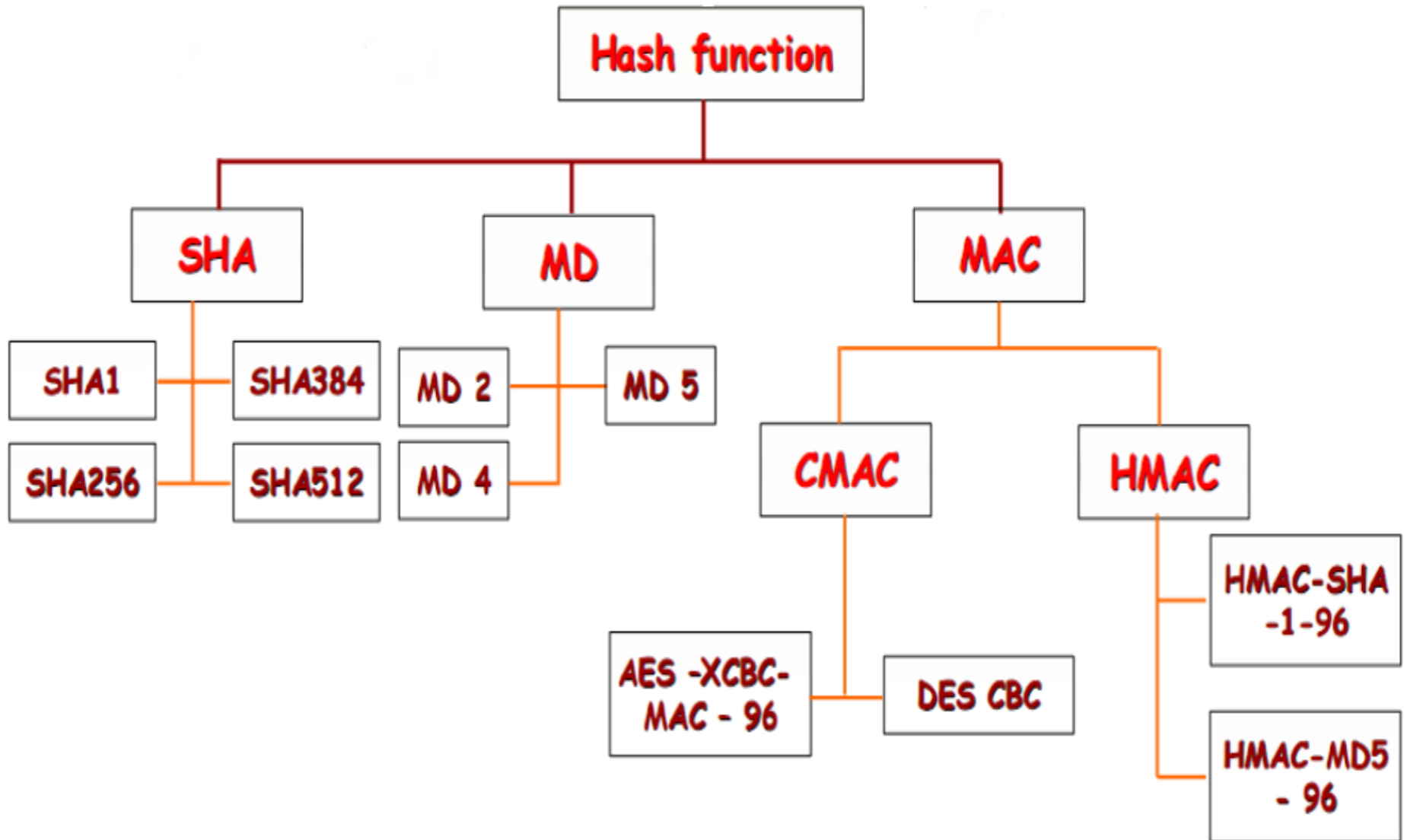
The word "cats" will be hashed to:  
 $3 + 1 + 20 + 19 = 43$



# Requirements for hash functions

- **Fast to compute** —  $O(1)$  time, low memory usage.
- **Deterministic** — the same key always produces the same hash value.
- **Uniform distribution** — keys should map evenly across the table to minimize collisions.
- **Low collision rate** — different keys should rarely map to the same index.
- **One-way (for cryptographic use)** — it should be computationally infeasible to reverse the hash back to the original key.

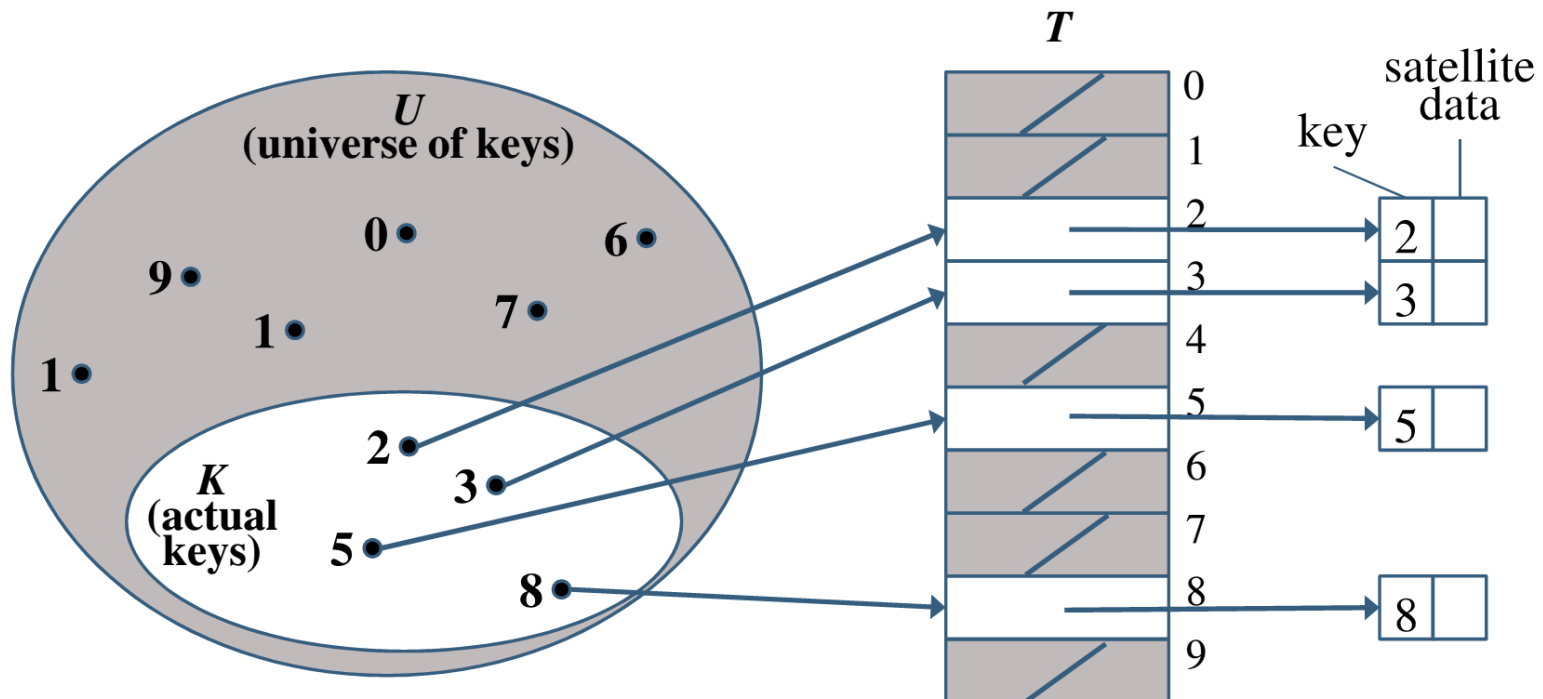
# Some famous hash functions



# Application of hashing

- Search with  $O(1)$

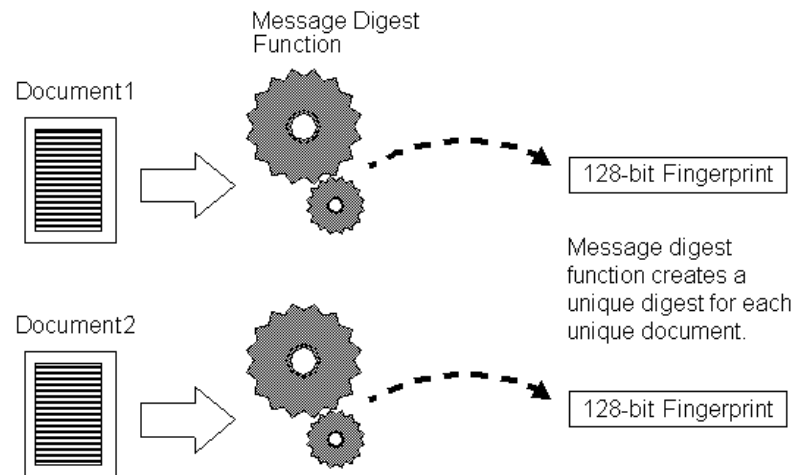
- Each element's key / value in the array will be hashing to serve as the address to store the element.
- When looking for an element, just hashing back, we will get the address and jump to its location.



# Application of hashing

- Check file integrity

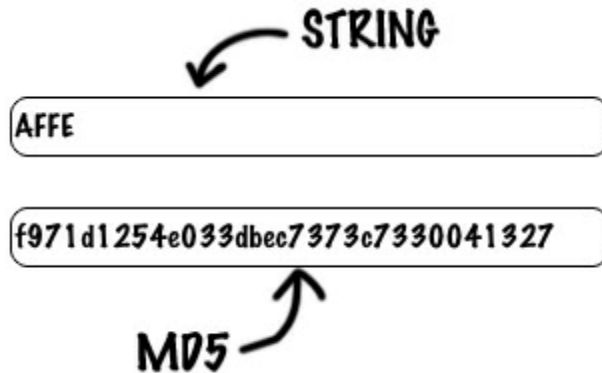
- The two files are hashed into 2 strings. If the strings are identical, the two files are identical.
- Application:
  - Check if the file transferred on the network is missing any bits
  - Whether the text transmitted over the network has been edited





# Application of hashing

- Encode/Decode:
  - Secure password storage.
  - Electronic Signature
  - Use on encryption algorithms



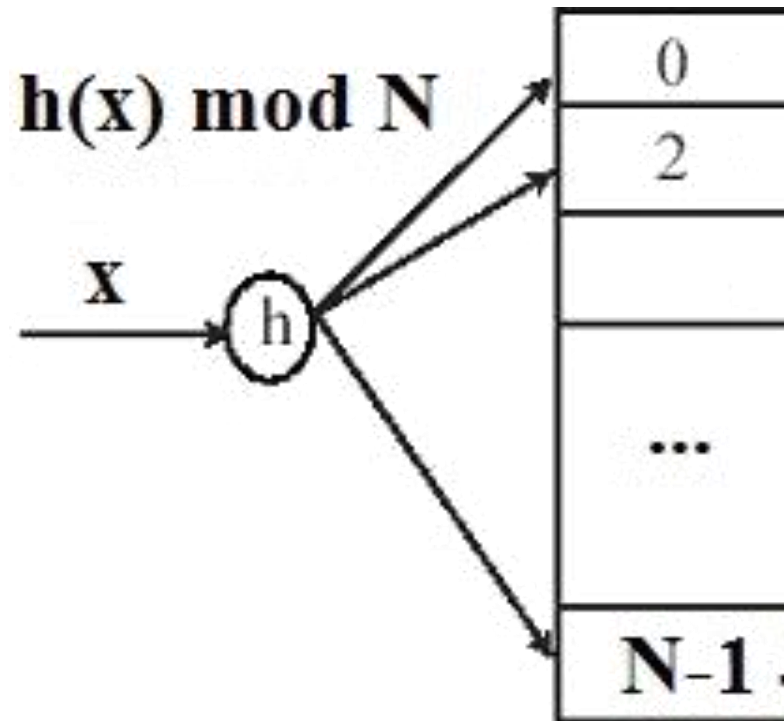
=  
79054025  
255fb1a2  
6e4bc422  
aef54eb4

# Implementing hash function

- Choose the Hash function :

$$H(x) = x \bmod N$$

where  $x$  is the element to add / find  
 $N$  is the array size



# Implementing hash function

- $N = 15$ :

|          |    |     |    |      |    |    |
|----------|----|-----|----|------|----|----|
| if $x =$ | 25 | 129 | 35 | 2501 | 47 | 36 |
| $H(x) =$ | 10 | 9   | 5  | 11   | 2  | 6  |

- The element will be contained in the array:

|   |   |    |   |   |    |    |   |   |     |    |      |    |    |    |
|---|---|----|---|---|----|----|---|---|-----|----|------|----|----|----|
| 0 | 1 | 2  | 3 | 4 | 5  | 6  | 7 | 8 | 9   | 10 | 11   | 12 | 13 | 14 |
| — | — | 47 | — | — | 35 | 36 | — | — | 129 | 25 | 2501 | —  | —  | —  |

- Adding, deleting, and editing only costs  $O(1)$  but ...

# Collision

- What happens if  $x = 65$ ?

$$x = 65$$

$$H(x) = 5$$

| 0 | 1 | 2  | 3 | 4 | 5  | 6  | 7 | 8 | 9   | 10 | 11   | 12 | 13 | 14 |
|---|---|----|---|---|----|----|---|---|-----|----|------|----|----|----|
| — | — | 47 | — | — | 35 | 36 | — | — | 129 | 25 | 2501 | —  | —  | —  |



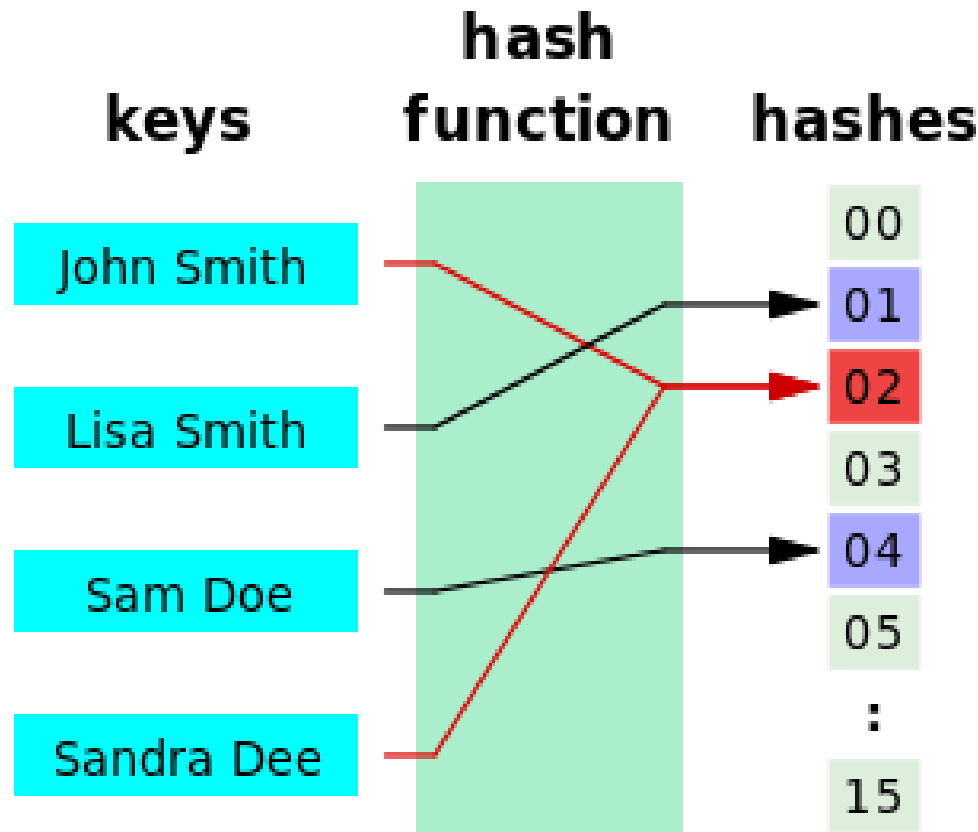
| 0 | 1 | 2  | 3 | 4 | 5      | 6  | 7 | 8 | 9   | 10 | 11   | 12 | 13 | 14 |
|---|---|----|---|---|--------|----|---|---|-----|----|------|----|----|----|
| — | — | 47 | — | — | 35     | 36 | — | — | 129 | 25 | 2501 | —  | —  | —  |
|   |   |    |   |   | 65 (?) |    |   |   |     |    |      |    |    |    |

→ Collision

# Collision

- *Collision* is the phenomenon of two or more keys being hashed into the same place.

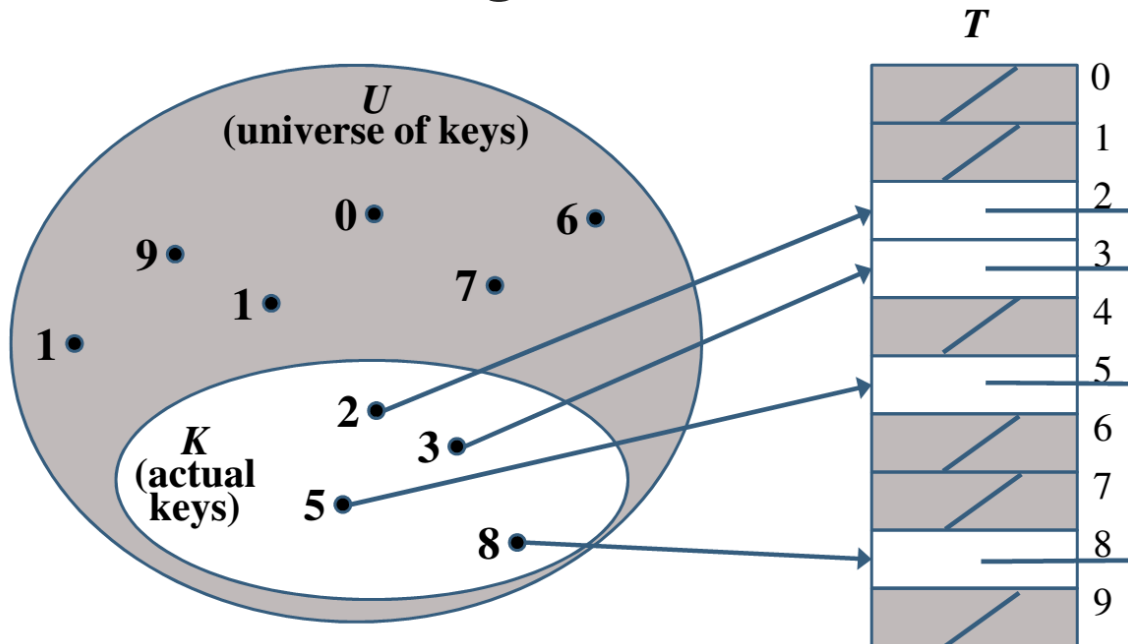
$$\exists k_1, k_2 \in K: k_1 \neq k_2, H(k_1) = H(k_2)$$



# The cause of the collision

Given a key set  $K$  to be stored:

- If  $|K| \leq N$ , whether the collision can happen depends on the hash function.
- If  $|K| > N$ , the collision is sure to happen no matter how good a hash function is.



The set of possible keys ( $U$ ) is often much larger than the actual key ( $K$ ).

# Load Factor

- The **load factor**  $\alpha$  measures how full the hash table is:  
$$\alpha = (\text{number of stored elements})/N$$
  - $\alpha$  close to 0: many empty slots, low collision risk, but wastes memory.
  - $\alpha$  close to 1: nearly full, high collision risk, degrades performance.
- **Rule of thumb:** Keep  $\alpha \leq 0.7$  for good performance.

# Collision Handling

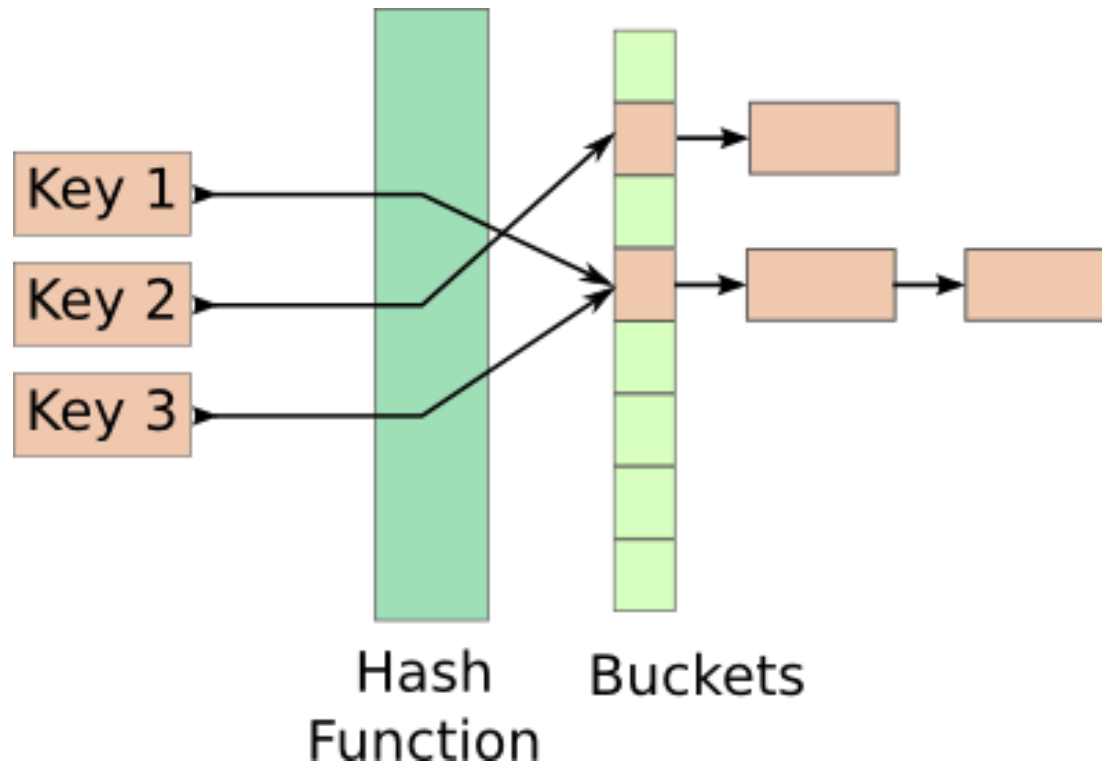
- Chaining approach
- Open-Addressing approach
  - Linear Probing
  - Quadratic Probing
  - Double Hashing



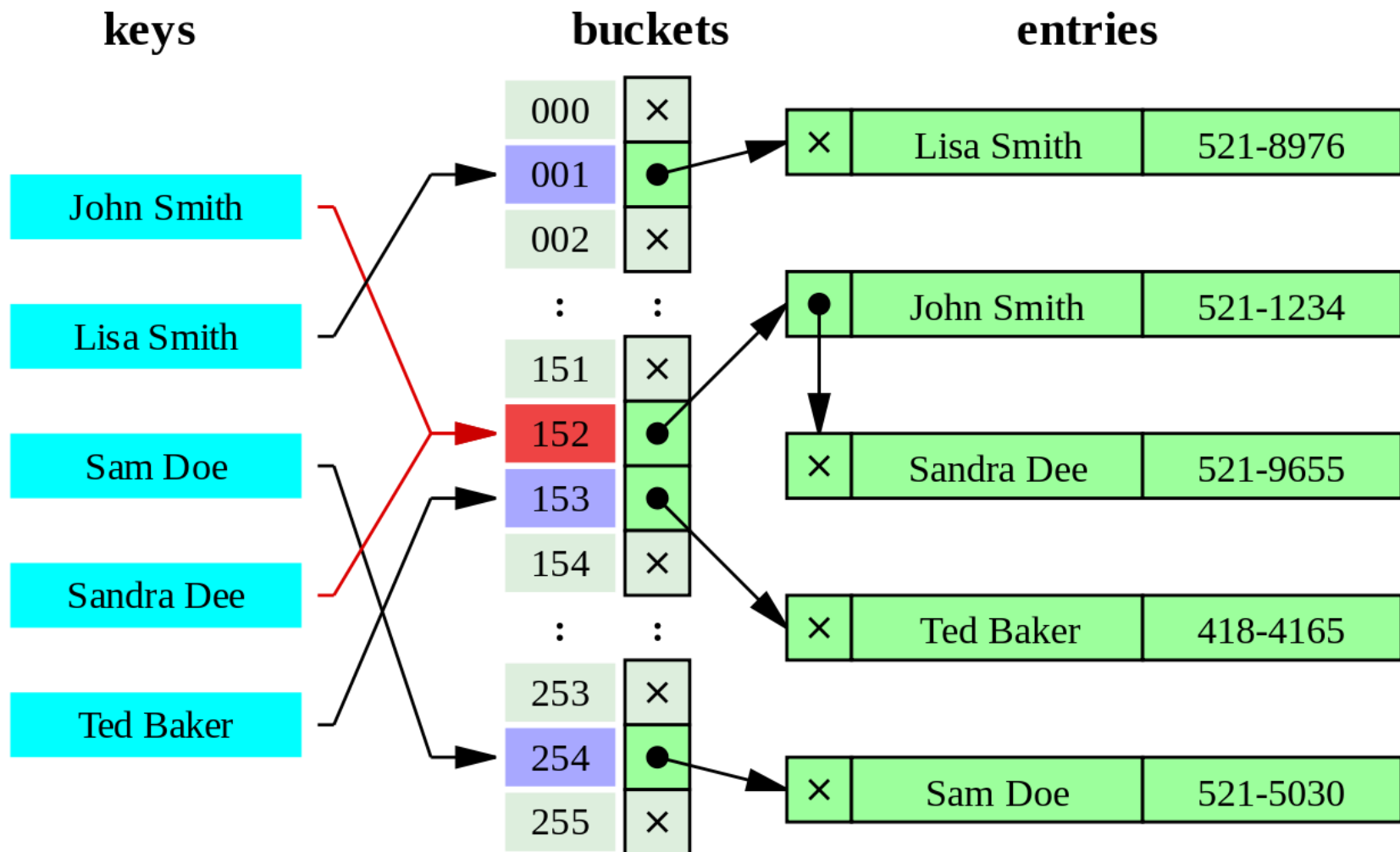


# Collision Handling - Chaining

- For each address of the table, we use the linked list to contain the collision key.
- From there, we have a hash table containing the headers of the linked lists.



# Chaining





|   |   |    |   |   |    |    |   |   |     |    |      |    |    |    |
|---|---|----|---|---|----|----|---|---|-----|----|------|----|----|----|
| 0 | 1 | 2  | 3 | 4 | 5  | 6  | 7 | 8 | 9   | 10 | 11   | 12 | 13 | 14 |
|   |   | ↓  |   |   | ↓  | ↓  |   |   | ↓   | ↓  | ↓    |    |    |    |
|   |   | 47 |   |   | 65 | 36 |   |   | 129 | 25 | 2501 |    |    |    |
|   |   |    |   |   | ↓  |    |   |   |     |    |      |    |    |    |
|   |   |    |   |   | 35 |    |   |   |     |    |      |    |    |    |

- Let's add the following keys to the above array using the hash table:

29, 16, 14, 99, 127

|   |    |    |   |   |    |    |     |   |     |    |      |    |    |    |
|---|----|----|---|---|----|----|-----|---|-----|----|------|----|----|----|
| 0 | 1  | 2  | 3 | 4 | 5  | 6  | 7   | 8 | 9   | 10 | 11   | 12 | 13 | 14 |
|   | ↓  | ↓  |   |   | ↓  | ↓  | ↓   |   | ↓   | ↓  | ↓    |    |    | ↓  |
|   | 16 | 47 |   |   | 65 | 36 | 127 |   | 99  | 25 | 2501 |    |    | 14 |
|   |    |    |   |   | ↓  |    |     |   | ↓   |    |      |    |    | ↓  |
|   |    |    |   |   | 35 |    |     |   | 129 |    |      |    |    | 29 |

# Comments

## Pros:

- Simple to implement.
- Table never becomes "full" — you can always add more elements.
- Deletion is straightforward.

## Cons:

- Uses extra memory for pointers (linked list nodes).
- Cache-unfriendly (pointers scattered in memory).
- Empty slots in array are wasted if chains grow long.
- Worst case degrades to  $O(n)$  if all keys hash to the same slot.

# Collision Handling

- Chaining approach
- Open-Addressing approach
  - Linear Probing
  - Quadratic Probing
  - Double Hashing



# Linear Probing

- Idea:
  - If the current position has been taken, we will find another position in the empty array.

- Linear probing:

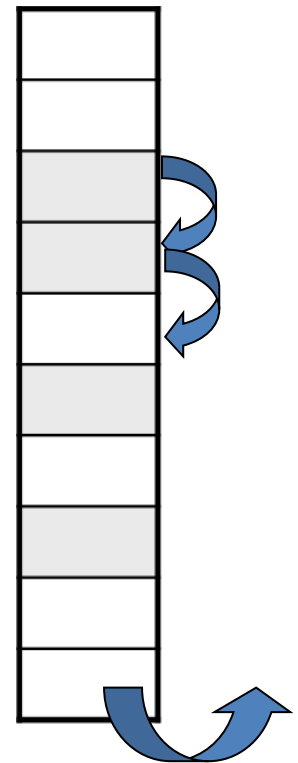
$$h(k,i) = (H(k) + i) \bmod N$$

where  $i$ : is the order of the attempt ( $i=0, 1, 2, \dots$ )

$H(k)$ : hash function

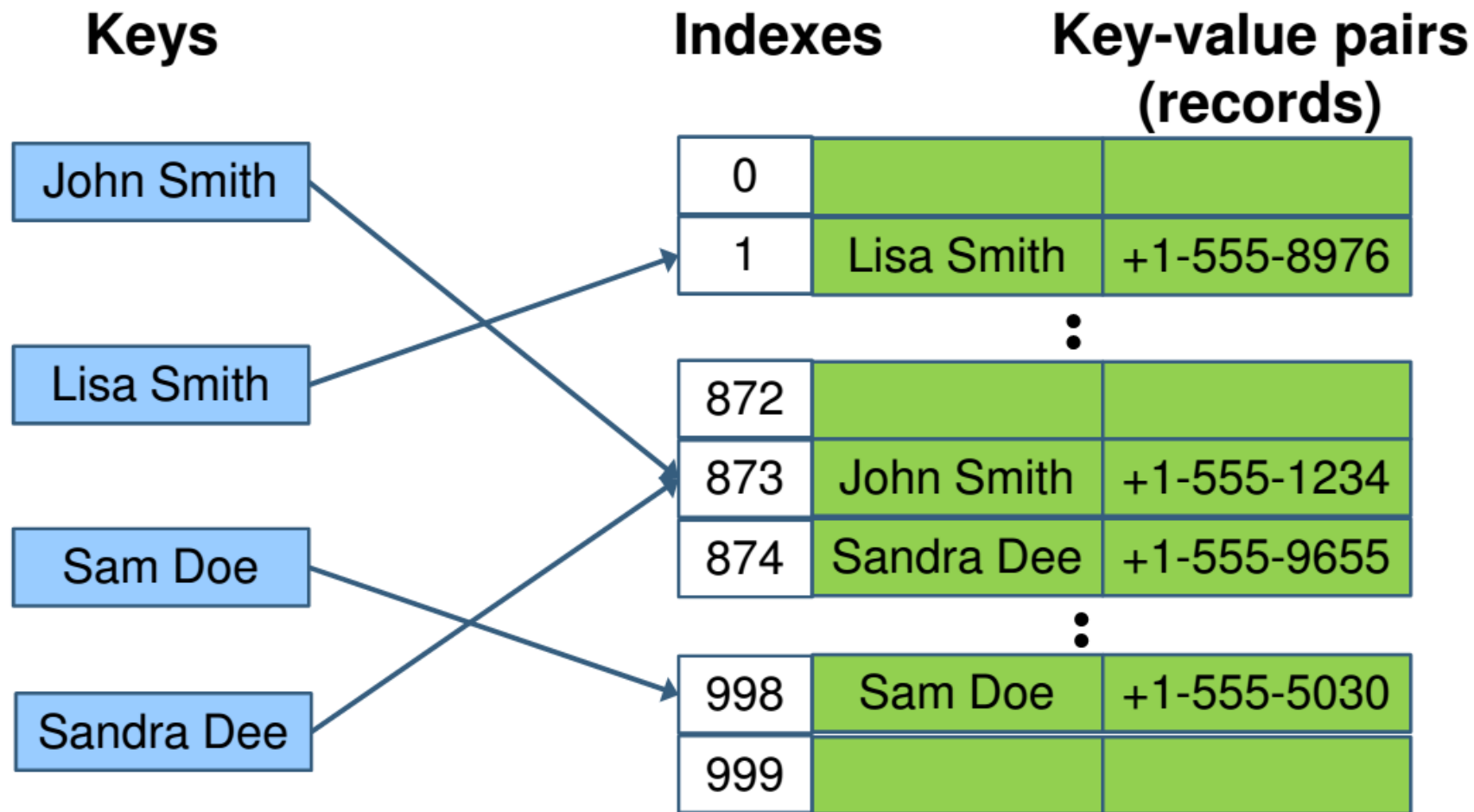
$N$ : the number of elements in the array.

Example:  $H(k), H(k) + 1, H(k) + 2, \dots$



wrap around

# Linear Probing



# Example

- Please use the open address method (linear probing) to resolve the following collision:

|   |   |    |   |   |       |    |   |   |     |    |      |    |    |    |
|---|---|----|---|---|-------|----|---|---|-----|----|------|----|----|----|
| 0 | 1 | 2  | 3 | 4 | 5     | 6  | 7 | 8 | 9   | 10 | 11   | 12 | 13 | 14 |
|   |   | 47 |   |   | 35    | 36 |   |   | 129 | 25 | 2501 |    |    |    |
|   |   |    |   |   | 65(?) |    |   |   |     |    |      |    |    |    |

- Answer:

|   |   |    |   |   |          |    |    |   |     |    |      |    |    |    |
|---|---|----|---|---|----------|----|----|---|-----|----|------|----|----|----|
| 0 | 1 | 2  | 3 | 4 | 5        | 6  | 7  | 8 | 9   | 10 | 11   | 12 | 13 | 14 |
|   |   | 47 |   |   | 35       | 36 | 65 |   | 129 | 25 | 2501 |    |    |    |
|   |   |    |   |   | ↑        | ↑  | ↑  |   |     |    |      |    |    |    |
|   |   |    |   |   | attempts |    |    |   |     |    |      |    |    |    |



# Example

- Please insert the following elements: 29, 16, 14, 99, 127 into the array.

|   |   |    |   |   |    |    |    |   |     |    |      |    |    |    |
|---|---|----|---|---|----|----|----|---|-----|----|------|----|----|----|
| 0 | 1 | 2  | 3 | 4 | 5  | 6  | 7  | 8 | 9   | 10 | 11   | 12 | 13 | 14 |
|   |   | 47 |   |   | 35 | 36 | 65 |   | 129 | 25 | 2501 |    |    |    |

- Answer:

|   |    |    |   |   |    |    |    |     |     |    |      |    |    |    |
|---|----|----|---|---|----|----|----|-----|-----|----|------|----|----|----|
| 0 | 1  | 2  | 3 | 4 | 5  | 6  | 7  | 8   | 9   | 10 | 11   | 12 | 13 | 14 |
|   | 16 | 47 |   |   | 35 | 36 | 65 | 127 | 129 | 25 | 2501 | 29 | 99 | 14 |

# Deletion Problem

- **You cannot simply remove an element!** Doing so breaks the probe chain for later elements.
- **Solution:** Mark deleted slots with a special **tombstone** marker (Ex: -2)
- During search, skip tombstones; during insert, reuse them.

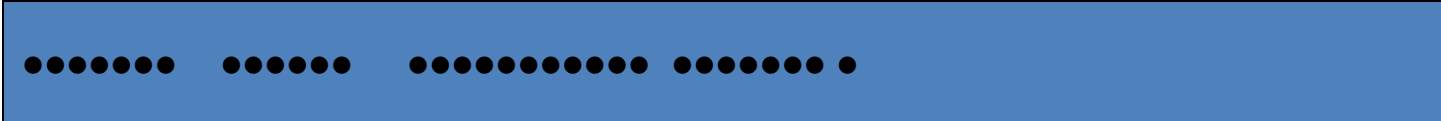
# Example

- Hash function:  $H = k \bmod 10$
- Please perform the following operations:
  - Add 47, 57, 68, 18, 67
  - Search 68
  - Search 10
  - Delete 47
  - Search 57
  - Insert: 77

|   |  |
|---|--|
| 0 |  |
| 1 |  |
| 2 |  |
| 3 |  |
| 4 |  |
| 5 |  |
| 6 |  |
| 7 |  |
| 8 |  |
| 9 |  |

# Comments

- Advantages:
  - There is no cost of creating a node like the chaining method.
  - Use of free space.
- Disadvantages:
  - The search will cost
  - Deletion will be problematic.
  - Distribution of elements is often in the form of groups, leading to heavy usage and unused space.



.....    .....    .....    .....    .

# Clustering Problem (Primary Clustering)

- Linear probing causes **primary clustering** — long consecutive runs of occupied slots.
- Once a cluster forms, future insertions are more likely to land in it, making it even longer. This degrades performance.
- **Analysis**

| Operation     | Average ( $\alpha < 0.5$ ) | Worst Case |
|---------------|----------------------------|------------|
| Insert        | $O(1)$                     | $O(n)$     |
| Search (hit)  | $O(1)$                     | $O(n)$     |
| Search (miss) | $O(1/(1-\alpha)^2)$        | $O(n)$     |

# Collision Handling

- Chaining approach
- Open-Addressing approach
  - Linear Probing
  - Quadratic Probing
  - Double Hashing



# Quadratic Probing

- Similar to linear probing but use the quadratic function to hope the distribution of the elements will be more evenly distributed:

$$h(k,i) = (H(k) + i^2) \bmod N$$

where  $i$ : is the order of the attempt ( $i=0, 1, 2, \dots$ )

$H(k)$ : hash function

$N$ : number of elements.

Example:  $H(k), H(k) + 1^2, H(k) + 2^2, \dots$

# Example

- Collision:

|   |   |    |   |   |        |    |   |   |     |    |      |    |    |    |
|---|---|----|---|---|--------|----|---|---|-----|----|------|----|----|----|
| 0 | 1 | 2  | 3 | 4 | 5      | 6  | 7 | 8 | 9   | 10 | 11   | 12 | 13 | 14 |
|   |   | 47 |   |   | 35     | 36 |   |   | 129 | 25 | 2501 |    |    |    |
|   |   |    |   |   | 65 (?) |    |   |   |     |    |      |    |    |    |

- Answer:

|   |   |    |   |   |          |     |   |   |     |    |      |    |    |     |
|---|---|----|---|---|----------|-----|---|---|-----|----|------|----|----|-----|
| 0 | 1 | 2  | 3 | 4 | 5        | 6   | 7 | 8 | 9   | 10 | 11   | 12 | 13 | 14  |
|   |   | 47 |   |   | 35       | 36  |   |   | 129 | 25 | 2501 |    |    | 65  |
|   |   |    |   |   | ↑        | ↑   |   |   | ↑   |    |      |    |    | ↑   |
|   |   |    |   |   | t        | t+1 |   |   | t+4 |    |      |    |    | t+9 |
|   |   |    |   |   | attempts |     |   |   |     |    |      |    |    |     |



# Example

- Continue adding the elements: 16, 14, 99, and 127 to the array.

|   |   |    |   |   |    |    |   |   |     |    |      |    |    |    |
|---|---|----|---|---|----|----|---|---|-----|----|------|----|----|----|
| 0 | 1 | 2  | 3 | 4 | 5  | 6  | 7 | 8 | 9   | 10 | 11   | 12 | 13 | 14 |
|   |   | 47 |   |   | 35 | 36 |   |   | 129 | 25 | 2501 |    |    | 65 |

- Answer:

|    |    |    |    |   |    |    |     |   |     |    |      |    |    |    |
|----|----|----|----|---|----|----|-----|---|-----|----|------|----|----|----|
| 0  | 1  | 2  | 3  | 4 | 5  | 6  | 7   | 8 | 9   | 10 | 11   | 12 | 13 | 14 |
| 29 | 16 | 47 | 14 |   | 35 | 36 | 127 |   | 129 | 25 | 2501 |    | 99 | 65 |

# Comments

- Advantage:
  - Element distribution is more even than linear probing (more evenly spread rather than concentrated in one place, but not too good either).
- Disadvantage:
  - Secondary clustering — two keys with the same  $H(k)$  follow the exact same probe sequence.
  - It is possible to fall into the endless loop if precautions are not taken.
    - Example: Try adding key 16 to the array of size 16. Know that positions 0, 1, 4, 9 already contain elements
    - **The array size should be prime.**

# Quiz

- Given the keys as 3,7,17,33,47,9,26,14,13,23,50,40. Hash function is  $H(x) = x \% 10$ . Explain the hashing process using linear probing and Quadratic probing.

# Quiz

- Using closed hashing, with double hashing to resolve collisions, insert the following keys into a hash table of thirteen slots (the slots are numbered 0 through 12). The hash functions to be used are H1 and H2, defined below. You should show the hash table after all eight keys have been inserted. Be sure to indicate how you are using H1 and H2 to do the hashing. Function  $\text{Rev}(k)$  reverses the decimal digits of  $k$ , for example,  $\text{Rev}(37) = 73$ ;  $\text{Rev}(7) = 7$ .

$$H1(k) = k \bmod 13.$$

$$H2(k) = (\text{Rev}(k + 1) \bmod 11).$$

Keys: 2, 8, 31, 20, 19, 18, 53, 27.

# Collision Handling

- Chaining approach
- Open-Addressing approach
  - Linear Probing
  - Quadratic Probing
  - Double Hashing



# Double Hashing

- Use two hash functions:
  - The first function to locate the jump to.
  - The second function determines the jump if there is a collision.
- Double hash method:

$$h(k,i) = [H_1(k) + i * H_2(k)] \bmod N$$

where  $i$ : is the order of the attempt ( $i=0, 1, 2, \dots$ )

$H_1(k)$ : first hash function

$H_2(k)$ : second hash function

$N$ : number of elements.

Example:  $H(k), H(k) + 1^2, H(k) + 2^2, \dots$

# Double Hashing

- Suppose the hash functions are as follows:

$$H_1(k) = k \bmod 13$$

$$H_2(k) = 1 + (k \bmod 11)$$

$$h(k,i) = [H_1(k) + i * H_2(k)] \bmod 13$$

- Adds element 14 to the array:

$$h_1(14,0) = 14 \bmod 13 = 1$$

$$\begin{aligned} h(14,1) &= (h_1(14) + h_2(14)) \bmod 13 \\ &= (1 + 4) \bmod 13 = 5 \end{aligned}$$

$$\begin{aligned} h(14,2) &= (h_1(14) + 2 h_2(14)) \bmod 13 \\ &= (1 + 8) \bmod 13 = 9 \end{aligned}$$

|    |    |
|----|----|
| 0  |    |
| 1  | 79 |
| 2  |    |
| 3  |    |
| 4  | 69 |
| 5  | 98 |
| 6  |    |
| 7  | 72 |
| 8  |    |
| 9  | 14 |
| 10 |    |
| 11 | 50 |
| 12 |    |

# The second hash function

- Usually, the second hash function has the form :

$$H_2(k) = R - (k \bmod R)$$

where  $R$  is prime,  $R < N$

- This ensures  $H_2(k) \neq 0$  (never gets stuck at the same slot).



# Example

- Let the following hash functions:

$$H_1(k) = k \bmod 15$$

$$H_2(k) = 11 - (k \bmod 11)$$

|   |   |    |   |   |       |    |   |   |     |    |      |    |    |    |
|---|---|----|---|---|-------|----|---|---|-----|----|------|----|----|----|
| 0 | 1 | 2  | 3 | 4 | 5     | 6  | 7 | 8 | 9   | 10 | 11   | 12 | 13 | 14 |
|   |   | 47 |   |   | 35    | 36 |   |   | 129 | 25 | 2501 |    |    |    |
|   |   |    |   |   | 65(?) |    |   |   |     |    |      |    |    |    |

- Answer:

|   |   |    |   |   |    |     |     |   |     |    |      |    |    |    |
|---|---|----|---|---|----|-----|-----|---|-----|----|------|----|----|----|
| 0 | 1 | 2  | 3 | 4 | 5  | 6   | 7   | 8 | 9   | 10 | 11   | 12 | 13 | 14 |
|   |   | 47 |   |   | 35 | 36  | 65  |   | 129 | 25 | 2501 |    |    |    |
|   |   |    |   |   | ↑  | ↑   | ↑   |   |     |    |      |    |    |    |
|   |   |    |   |   | t  | t+1 | t+2 |   |     |    |      |    |    |    |

# Example

- Let's continue adding the following elements to the array: 16, 14, 99, 127 (!)

|   |   |    |   |   |    |    |    |   |     |    |      |    |    |    |
|---|---|----|---|---|----|----|----|---|-----|----|------|----|----|----|
| 0 | 1 | 2  | 3 | 4 | 5  | 6  | 7  | 8 | 9   | 10 | 11   | 12 | 13 | 14 |
|   |   | 47 |   |   | 35 | 36 | 65 |   | 129 | 25 | 2501 |    |    |    |

- Answer:

|    |    |    |   |   |    |    |    |   |     |    |      |    |    |    |
|----|----|----|---|---|----|----|----|---|-----|----|------|----|----|----|
| 0  | 1  | 2  | 3 | 4 | 5  | 6  | 7  | 8 | 9   | 10 | 11   | 12 | 13 | 14 |
| 14 | 16 | 47 |   |   | 35 | 36 | 65 |   | 129 | 25 | 2501 | 99 |    | 29 |

*Element 127: infinite loop*

# Comments

- Advantages:
  - Eliminates both primary and secondary clustering.
  - The most uniform distribution among open addressing methods.
- Disadvantages:
  - Execution is slower than quadratic probing because the second hash is computed.
  - Choosing good H1 and H2 requires care.

# Comparison of Collision Handling Methods

| Feature           | Chaining              | Linear Probing                   | Quadratic Probing    | Double Hashing        |
|-------------------|-----------------------|----------------------------------|----------------------|-----------------------|
| Memory            | Extra (pointers)      | In-table only                    | In-table only        | In-table only         |
| Clustering        | None                  | Primary clustering               | Secondary clustering | Minimal               |
| Deletion          | Easy                  | Needs tombstone                  | Needs tombstone      | Needs tombstone       |
| Load factor limit | $> 1$ allowed         | $\leq 1$ (typically $\leq 0.7$ ) | $\leq 0.5$           | $\leq 0.7$            |
| Speed             | Fast, cache-miss risk | Fast (cache friendly)            | Moderate             | Slower (2 hash calls) |
| Complexity        | Moderate              | Simple                           | Moderate             | More complex          |

# When to use which?

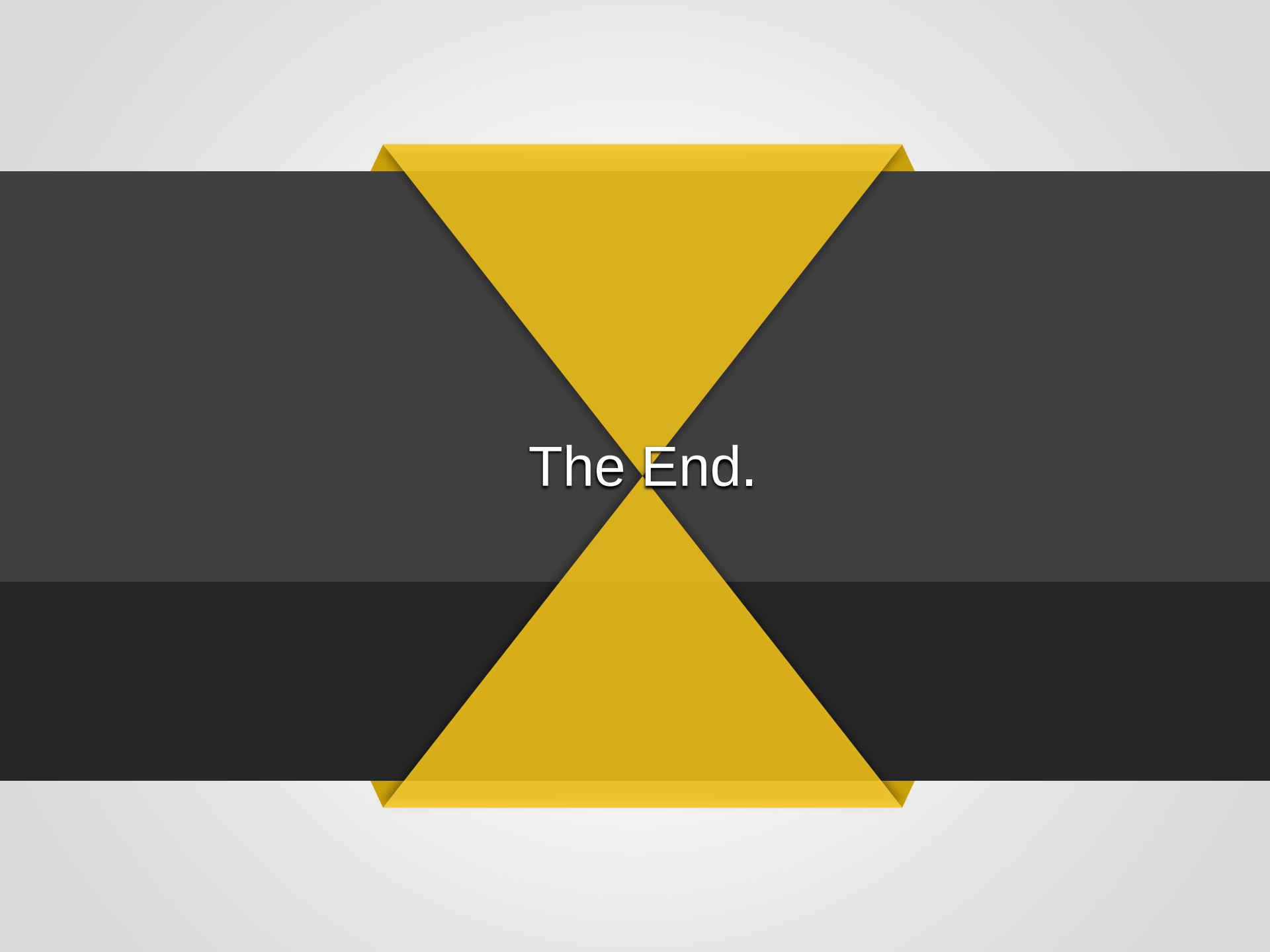
- **Chaining** — when the number of elements is unknown or unbounded; easier to implement.
- **Linear probing** — when memory is tight and cache performance matters; keep  $\alpha$  low.
- **Quadratic probing** — a middle-ground; remember to use prime  $N$  and  $\alpha \leq 0.5$ .
- **Double hashing** — when you need the best distribution with open addressing.

# Quiz

- Write a function that accepts a string that contains all the letters of the alphabet except one and returns the missing letter. For example, the string, "the quick brown box jumps over a lazy dog" contains all the letters of the alphabet except the letter, "f". The function should have a time complexity of  $O(N)$ .

# Advanced topics

- Other search algorithms:
  - Search algorithms on graphs
  - Search with Heuristic
  - Hill climbing
  - ...

A large, stylized yellow 'X' shape is centered on a dark gray background. The 'X' is composed of two overlapping triangles, with a slight 3D effect suggested by a darker yellow shadow on the right side of each triangle. The text 'The End.' is written in a white, sans-serif font, centered within the intersection of the 'X'.

The End.